

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Факультет безопасности информационных технологий

Дисциплина:

«Информатика»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

Основные принципы функционирования вычислительных устройств

Выполнил:

Студент группы N3150

Пахомов Владимир Юрьевич



Проверил:

Грозов В.А.

Санкт-Петербург

2024г.

Оглавление

Задание. Вариант 5-9.....	3
Make-файл.....	3
Примеры работы программ на различных исходных данных.....	3
Дизассемблерный листинг. Выводы.....	4
Исходные тексты программ с комментариями.....	5

Задание. Вариант 5-9

Тип числа: uint16_t

Задание: Назовём свёрткой байта порядка N операцию циклического сдвига старшей тетрады на N битов вправо, а младшей тетрады на N битов влево. Выполнить свёртку всех байтов на случайное число из диапазона 0..3.

Пример преобразования: 0xDEADBEEF => 0x7BA7EBBF (N = 2)

Make-файл

.PHONY: all clean debug

APP=inf2vypN3150

CFLAGS=-Wall -Wextra -Werror

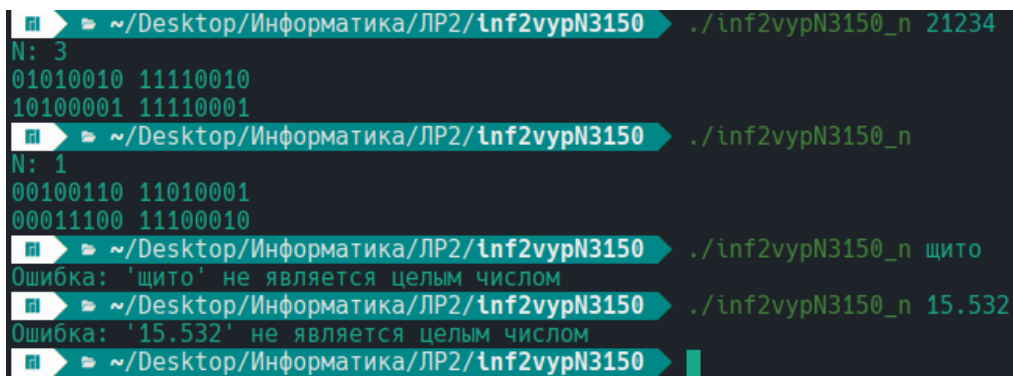
all: \$(APP)

```
$(APP): $(APP).c $(APP).S
gcc -o $(APP)_c $(CFLAGS) $(APP).c
nasm -felf64 $(APP).S
gcc -no-pie -o $(APP)_n $(APP).o
```

clean:

```
rm -rf $(APP)_c $(APP)_n $(APP).o
```

Примеры работы программ на различных исходных данных



```
~/Desktop/Информатика/ЛП2/inf2vypN3150 ➤ ./inf2vypN3150_n 21234
N: 3
01010010 11110010
10100001 11110001
~/Desktop/Информатика/ЛП2/inf2vypN3150 ➤ ./inf2vypN3150_n
N: 1
00100110 11010001
00011100 11100010
~/Desktop/Информатика/ЛП2/inf2vypN3150 ➤ ./inf2vypN3150_n щито
Ошибка: 'щито' не является целым числом
~/Desktop/Информатика/ЛП2/inf2vypN3150 ➤ ./inf2vypN3150_n 15.532
Ошибка: '15.532' не является целым числом
~/Desktop/Информатика/ЛП2/inf2vypN3150 ➤
```

Дизассемблерный листинг. Выводы

Функция преобразования числа (слева C, справа NASM):

```
000000000000124d <circular_shift>:
124d: 55             push rbp
124e: 48 89 e5       mov rbp,rbp
1251: 89 f8         mov eax,edi
1253: 89 75 e8       mov DWORD PTR [rbp-0x18],esi
1256: 66 89 45 ec    mov WORD PTR [rbp-0x14],ax
125a: 66 c7 45 f8 00 00 mov WORD PTR [rbp-0x8],0x0
1260: c7 45 fc 00 00 00 mov DWORD PTR [rbp-0x4],0x0
1267: e9 ab 00 00 00 jmp 1317 <circular_shift+0xca>
126c: 0f b7 55 ec    movzx edx,WORD PTR [rbp-0x14]
1270: 8b 45 fc       mov eax,DWORD PTR [rbp-0x4]
1273: c1 e0 03       shl eax,0x3
1276: 89 c1         mov ecx,eax
1278: d3 fa         sar edx,cl
127a: 89 d0         mov eax,edx
127c: 88 45 f6       mov BYTE PTR [rbp-0xa],al
127f: 0f b6 45 f6    movzx eax,BYTE PTR [rbp-0xa]
1283: c8 e8 04       shr al,0x4
1286: 88 45 f7       mov BYTE PTR [rbp-0x9],al
1289: 0f b6 45 f6    movzx eax,BYTE PTR [rbp-0xa]
128c: 83 e0 0f       and eax,0xf
128e: 88 45 f8       mov BYTE PTR [rbp-0x8],al
1293: 0f b6 55 f7    movzx edx,BYTE PTR [rbp-0x9]
1297: 8b 45 e8       mov eax,DWORD PTR [rbp-0x18]
129a: 89 c1         mov ecx,eax
129c: d3 fa         sar edx,cl
129e: 89 d0         mov eax,edx
12a0: 89 c0         mov esi,eax
12a2: 0f b6 55 f7    movzx edx,BYTE PTR [rbp-0x9]
12a6: b8 04 00 00 00 mov eax,0x4
12ab: 2b 45 e8       sub eax,DWORD PTR [rbp-0x18]
12ae: 89 c1         mov ecx,eax
12b0: d3 c1         shl edx,cl
12b2: 89 d0         mov eax,edx
12b4: 89 f8         mov esi,eax
12b6: 83 e0 0f       and eax,0xf
12b9: 88 45 f7       mov BYTE PTR [rbp-0x9],al
12bc: 0f b6 55 f8    movzx edx,BYTE PTR [rbp-0x8]
12c0: 8b 45 e8       mov eax,DWORD PTR [rbp-0x18]
12c3: 89 c1         mov ecx,eax
12c5: d3 e2         shl edx,cl
12c7: 89 d0         mov eax,edx
12c9: 89 c0         mov esi,eax
12cb: 0f b6 55 f8    movzx edx,BYTE PTR [rbp-0x8]
12cf: b8 04 00 00 00 mov eax,0x4
12d4: 2b 45 e8       sub eax,DWORD PTR [rbp-0x18]
12d7: 89 c1         mov ecx,eax
12d9: d3 fa         sar edx,cl
12db: 89 d0         mov eax,edx
12dd: 89 f8         mov esi,eax
12df: 83 e0 0f       and eax,0xf
12e2: 88 45 f9       mov BYTE PTR [rbp-0x8],al
12e5: 0f b6 45 f7    movzx eax,BYTE PTR [rbp-0x9]
12e9: c1 e0 04       shl eax,0x4
12ec: 89 c2         mov ecx,eax
12ee: 0f b6 45 f8    movzx eax,BYTE PTR [rbp-0x8]
12f2: 89 d0         mov eax,edx
12f4: 88 45 f9       mov BYTE PTR [rbp-0x7],al
12f7: 0f b6 55 f9    movzx edx,BYTE PTR [rbp-0x7]
12fb: 8b 45 fc       mov eax,DWORD PTR [rbp-0x4]
12fe: c1 e0 03       shl eax,0x3
1301: 89 c1         mov ecx,eax
1303: d3 e2         shl edx,cl
1305: 89 d0         mov eax,edx
1307: 89 c2         mov ecx,eax
1309: 0f b7 45 fa     movzx eax,WORD PTR [rbp-0x6]
130b: 89 d0         mov eax,edx
130f: 66 89 45 fa     mov WORD PTR [rbp-0x6],ax
1313: 83 45 fc 01     add DWORD PTR [rbp-0x4],0x1
1317: 83 7d fc 01     cmp DWORD PTR [rbp-0x4],0x1
131b: 0f 8e 40 ff ff jle 126c <circular_shift+0x1f>
131f: 0f b7 45 fa     movzx eax,WORD PTR [rbp-0x6]
1325: 5d             pop rbp
1326: c3             ret

000000000000401190 <circular_shift>:
401190: b8 00 00 00 00 mov eax,0x0
401192: 32 00         mov dl,0x0
401197: 41 b1 04       mov r9b,0x4
40119a: 41 28 f1       sub r9b,sil

00000000000040119d <circular_shift_loop>:
40119d: 80 fa 10       cmp dl,0x10
4011a0: 73 70         jae 401212 <circular_shift_exit>
4011a2: 66 41 89 fa    mov r10w,di
4011a6: 88 d1         mov cl,dl
4011a8: 66 41 d3 ea    shr r10w,cl
4011ac: 66 41 81 e2 ff 00 and r10w,0xffff
4011b2: 66 45 09 d3    mov r11w,r10w
4011b6: 66 41 c1 eb 04 shr r11w,0x4
4011bb: 66 45 89 d4    mov r12w,r10w
4011bf: 66 41 83 e4 0f and r12w,0xf
4011c4: 66 45 89 dd    mov r13w,r11w
4011c8: 40 88 f1       mov cl,sil
4011cb: 66 41 d3 ed    shr r13w,cl
4011cf: 44 00 c9       mov cl,r9b
4011d2: 66 41 d3 e3    shl r11w,cl
4011d6: 66 45 09 eb    or r11w,r13w
4011da: 66 41 83 e3 0f and r11w,0xf
4011de: 66 45 89 ee    mov r14w,r12w
4011e3: 40 88 f1       mov cl,sil
4011e6: 66 41 d3 e6    shl r14w,cl
4011ea: 44 88 c9       mov cl,r9b
4011ed: 66 41 d3 ec    shr r12w,cl
4011f1: 66 45 09 f4    or r12w,r14w
4011f5: 66 41 83 e4 0f and r12w,0xf
4011fa: 66 41 c1 e3 04 shl r11w,0x4
4011ff: 66 45 09 e3    or r10w,r12w
401203: 88 d1         mov cl,dl
401205: 66 41 d3 e3    shl r11w,cl
401209: 66 44 09 d0    mov ax,r11w
40120d: 80 c2 08       add dl,0x8
401210: eb 8b         jmp 40119d <circular_shift_loop>

000000000000401212 <circular_shift_exit>:
401212: c3             ret

000000000000401213 <print_bin>:
401213: 48 83 ec 03    sub rsp,0x3
401217: c6 04 24 10    mov BYTE PTR [rsp],0x10
40121b: 66 03 7c 24 01 mov WORD PTR [rsp+0x1],di

000000000000401220 <print_bin_loop>:
401220: 8a 0c 24       mov cl,BYTE PTR [rsp]
401223: fe c9         dec cl
401225: 88 0c 24       cmp cl,BYTE PTR [rsp],cl
401228: 0f 90 00 00 00 jmp 401201 <print_bin_loop_exit>
40122b: 7c 24         jll 401201 <print_bin_loop_exit>
40122d: 41 ba 00 00 00 00 mov r10d,0x0
401233: 66 44 8b 54 24 01 mov r10w,WORD PTR [rsp+0x1]
401239: 49 d3 ea       shr r10,cl
40123c: 49 d3 e2 01    and r10,0x1
401240: 48 bf a9 40 40 00 00 movabs rdi,0x4040a9
401247: 00 00 00       mov rsi,r10
40124a: 4c 89 d6       mov eax,r10
40124d: b8 00 00 00 00 mov eax,0x0
401252: e8 e9 fd ff ff call 401040 <printf@plt>
401257: 4c 0b 1c 24    mov r11,QWORD PTR [rsp]
40125b: 49 83 f0 00    cmp r11,0x0
40125f: 74 bf         je 401220 <print_bin_loop>
401261: 49 83 e3 07    and r11,0x7
401265: 49 83 f0 00    cmp r11,0x0
401269: 75 05         jne 401220 <print_bin_loop>
40126b: 48 bf ac 40 40 00 00 movabs rdi,0x4040ac
401272: 00 00 00       mov eax,0x0
401275: b8 00 00 00 00 mov eax,0x0
40127a: e8 2d fd ff ff call 401040 <printf@plt>
40127f: eb 9f         jmp 401220 <print_bin_loop>

000000000000401281 <print_bin_loop_exit>:
401281: 48 bf ae 40 40 00 00 movabs rdi,0x4040ae
401288: 00 00 00       mov eax,0x0
40128b: b8 00 00 00 00 mov eax,0x0
40128e: e8 2d fd ff ff call 401040 <printf@plt>
401295: add rsp,0x3
401299: c3             ret
```

Функция вывода числа в двоичном виде:

```
000000000000011d0 <print_bin>:
11d0: 55             push rbp
11d1: 48 89 e5       mov rbp,rbp
11d4: 48 83 ec 20    sub rsp,0x20
11d6: 89 f8         mov eax,edi
11d8: 89 75 e8       mov DWORD PTR [rbp-0x14],esi
11da: 66 89 45 ec    mov WORD PTR [rbp-0x14],ax
11de: c7 45 f8 0f 00 00 00 mov WORD PTR [rbp-0x8],0xf
11e2: e9 4a 00 00 00 jmp 123a <print_bin+0x61>
11e8: 0f b7 55 ec    movzx edx,WORD PTR [rbp-0x14]
11f4: 8b 45 f8       mov eax,DWORD PTR [rbp-0x8]
11f7: 89 c1         mov ecx,eax
11f9: d3 fa         sar edx,cl
11fb: 89 d0         mov eax,edx
11fd: 83 e0 01       and eax,0x1
11fe: 89 45 fc       mov DWORD PTR [rbp-0x4],eax
1203: 8b 45 fc       mov eax,DWORD PTR [rbp-0x4]
1206: 89 c0         mov esi,eax
1208: 48 05 f9 0d 00 00 00 lea rax,[rip+0xd9] # 2008 <_IO_stdin_used+0x8>
120f: 48 89 c7       mov r10,rax
1212: b8 00 00 00 00 mov eax,0x0
1217: e8 54 fe ff ff call 1070 <printf@plt>
121c: 8b 45 f8       mov eax,DWORD PTR [rbp-0x8]
121f: 83 e0 07       and eax,0x7
1222: 85 c0         test eax,eax
1224: 75 10         jne 1236 <print_bin+0x5d>
1226: 83 7d f8 00    cmp DWORD PTR [rbp-0x8],0x0
122a: 74 0a         jbe 1236 <print_bin+0x5d>
122c: bf 20 00 00 00 mov edi,0x20
1231: e8 0a fe ff ff call 1040 <putchar@plt>
1236: 83 6d f8 01    sub DWORD PTR [rbp-0x8],0x1
123a: 83 7d f8 00    cmp DWORD PTR [rbp-0x8],0x0
123e: 79 b0         jns 11f0 <print_bin+0x17>
1240: bf 0a 00 00 00 mov edi,0xa
1245: e8 f6 fd ff ff call 1040 <putchar@plt>
124a: 90             nop
124b: c9             leave ecx
124c: c3             ret

0000000000000124d <circular_shift>:
124d: 55             push rbp
124e: 48 89 e5       mov rbp,rbp
1251: 89 f8         mov eax,edi
1253: 89 75 e8       mov DWORD PTR [rbp-0x18],esi
1256: 66 89 45 ec    mov WORD PTR [rbp-0x14],ax
125a: 66 c7 45 f8 00 00 mov WORD PTR [rbp-0x8],0x0
1260: c7 45 fc 00 00 00 mov DWORD PTR [rbp-0x4],0x0
1267: e9 ab 00 00 00 jmp 1317 <circular_shift+0xca>
126c: 0f b7 55 ec    movzx edx,WORD PTR [rbp-0x14]
1270: 8b 45 fc       mov eax,DWORD PTR [rbp-0x4]
1273: c1 e0 03       shl eax,0x3
1276: 89 c1         mov ecx,eax
1278: d3 fa         sar edx,cl
127a: 89 d0         mov eax,edx
127c: 88 45 f6       mov BYTE PTR [rbp-0xa],al
127f: 0f b6 45 f6    movzx eax,BYTE PTR [rbp-0xa]
1283: c8 e8 04       shr al,0x4
1286: 88 45 f7       mov BYTE PTR [rbp-0x9],al
1289: 0f b6 45 f6    movzx eax,BYTE PTR [rbp-0xa]
128c: 83 e0 0f       and eax,0xf
128e: 88 45 f8       mov BYTE PTR [rbp-0x8],al
1293: 0f b6 55 f7    movzx edx,BYTE PTR [rbp-0x9]
1297: 8b 45 e8       mov eax,DWORD PTR [rbp-0x18]
129a: 89 c1         mov ecx,eax
129c: d3 fa         sar edx,cl
129e: 89 d0         mov eax,edx
12a0: 89 c0         mov esi,eax
12a2: 0f b6 55 f7    movzx edx,BYTE PTR [rbp-0x9]
12a6: b8 04 00 00 00 mov eax,0x4
12ab: 2b 45 e8       sub eax,DWORD PTR [rbp-0x18]
12ae: 89 c1         mov ecx,eax
12b0: d3 c1         shl edx,cl
12b2: 89 d0         mov eax,edx
12b4: 89 f8         mov esi,eax
12b6: 83 e0 0f       and eax,0xf
12b9: 88 45 f7       mov BYTE PTR [rbp-0x9],al
12bc: 0f b6 55 f8    movzx edx,BYTE PTR [rbp-0x8]
12c0: 8b 45 e8       mov eax,DWORD PTR [rbp-0x18]
12c3: 89 c1         mov ecx,eax
12c5: d3 e2         shl edx,cl
12c7: 89 d0         mov eax,edx
12c9: 89 c0         mov esi,eax
12cb: 0f b6 55 f8    movzx edx,BYTE PTR [rbp-0x8]
12cf: b8 04 00 00 00 mov eax,0x4
12d4: 2b 45 e8       sub eax,DWORD PTR [rbp-0x18]
12d7: 89 c1         mov ecx,eax
12d9: d3 fa         sar edx,cl
12db: 89 d0         mov eax,edx
12dd: 89 f8         mov esi,eax
12df: 83 e0 0f       and eax,0xf
12e2: 88 45 f9       mov BYTE PTR [rbp-0x8],al
12e5: 0f b6 45 f7    movzx eax,BYTE PTR [rbp-0x9]
12e9: c1 e0 04       shl eax,0x4
12ec: 89 c2         mov ecx,eax
12ee: 0f b6 45 f8    movzx eax,BYTE PTR [rbp-0x8]
12f2: 89 d0         mov eax,edx
12f4: 88 45 f9       mov BYTE PTR [rbp-0x7],al
12f7: 0f b6 55 f9    movzx edx,BYTE PTR [rbp-0x7]
12fb: 8b 45 fc       mov eax,DWORD PTR [rbp-0x4]
12fe: c1 e0 03       shl eax,0x3
1301: 89 c1         mov ecx,eax
1303: d3 e2         shl edx,cl
1305: 89 d0         mov eax,edx
1307: 89 c2         mov ecx,eax
1309: 0f b7 45 fa     movzx eax,WORD PTR [rbp-0x6]
130b: 89 d0         mov eax,edx
130f: 66 89 45 fa     mov WORD PTR [rbp-0x6],ax
1313: 83 45 fc 01     add DWORD PTR [rbp-0x4],0x1
1317: 83 7d fc 01     cmp DWORD PTR [rbp-0x4],0x1
131b: 0f 8e 40 ff ff jle 126c <circular_shift+0x1f>
131f: 0f b7 45 fa     movzx eax,WORD PTR [rbp-0x6]
1325: 5d             pop rbp
1326: c3             ret
```

Вывод: программа написанная на NASM выглядит короче и работает быстрее, чем программа дизассемблированная с C.

Исходные тексты программ с комментариями

Программа NASM:

```
global main

extern printf
extern fprintf
extern stderr

extern srand
extern rand

extern time

extern getpid

extern strtol

section .text

circular_shift:
    mov rax, 0
    mov dl, 0

    mov r9b, 4
    sub r9b, sil

    circular_shift_loop:
        cmp dl, 16
        jae circular_shift_exit

        mov r10w, di
        mov cl, dl
        shr r10w, cl
        and r10w, 0xFF

        mov r11w, r10w    ; high_tetr
        shr r11w, 4
        mov r12w, r10w    ; low_tetr
        and r12w, 0b1111

        mov r13w, r11w
        mov cl, sil
        shr r13w, cl
        mov cl, r9b
        shl r11w, cl
        or r11w, r13w
        and r11w, 0b1111

        mov r14w, r12w
        mov cl, sil
        shl r14w, cl
        mov cl, r9b
        shr r12w, cl
        or r12w, r14w
        and r12w, 0b1111

        shl r11w, 4
        or r11w, r12w
        mov cl, dl
        shl r11w, cl
        or ax, r11w
```

```

        add dl, 8
        jmp circular_shift_loop
circular_shift_exit:
ret

print_bin:
sub rsp, 3
mov byte [rsp], 16
mov [rsp+1], di

print_bin_loop:
mov cl, [rsp]
dec cl
mov [rsp], cl

    cmp cl, 0
    jl print_bin_loop_exit

    mov r10, 0
    mov r10w, [rsp+1]
    shr r10, cl
    and r10, 1
    mov rdi, numberstr
    mov rsi, r10
    mov rax, 0
    call printf

    mov r11, [rsp]
    cmp r11, 0
    je print_bin_loop
    and r11, 0b111
    cmp r11, 0
    jne print_bin_loop

    mov rdi, spacestr
    mov rax, 0
    call printf

    jmp print_bin_loop
print_bin_loop_exit:
mov rdi, lbstr
mov rax, 0
call printf

add rsp, 3
ret

main:
push rbp
mov rbp, rsp
sub rsp, 16

mov r12, rdi ; argc
mov r13, rsi ; argv

; word [rsp+1] - number
; byte [rsp] - N
sub rsp, 3

mov rdi, 0
mov rax, 0
call time
mov r10, rax

```

```

mov rax, 0
call getpid
add rax, r10

mov rdi, rax
mov rax, 0
call srand

mov rax, 0
call rand

and al, 0b11
mov byte [rsp], al

cmp r12, 1
je arg1
cmp r12, 2
je arg2

mov rdi, [stderr]
mov rsi, usagemsg
mov rdx, [r13]
mov rax, 0
call fprintf
jmp error

arg1:
    mov rax, 0
    call rand
    mov word [rsp+1], ax
    jmp argend

arg2:
    sub rsp, 8
    mov rdi, [r13+8]
    mov rsi, rsp
    mov rdx, 10
    mov rax, 0
    call strtol
    mov word [rsp+9], ax

    pop rax
    cmp rax, [r13+8]
    je arg2error

    mov al, [rax]
    cmp al, 0
    jne arg2error

    jmp argend

arg2error:
    mov rdi, [stderr]
    mov rsi, arg2errmsg
    mov rdx, [r13+8]
    mov rax, 0
    call fprintf
    jmp error

argend:

mov rdi, nmsg
mov rsi, 0

```

```

    mov sil, byte [rsp]
    mov rax, 0
    call printf

    mov rdi, 0
    mov di, word [rsp+1]
    call print_bin

    mov di, [rsp+1]
    mov sil, [rsp]
    call circular_shift
    mov [rsp+1], ax

    mov rdi, 0
    mov di, word [rsp+1]
    call print_bin

    add rsp, 3
    mov rax, 0
    leave
    ret
error:
    add rsp, 3
    mov rax, 1
    leave
    ret

section .data
    usagemsg      db      `Usage: %s [число]\n`, 0
    arg2errmsg    db      `Ошибка: '%s' не является целым числом\n`, 0
    nmsg          db      `N: %d\n`, 0
    numberstr     db      `%d`, 0
    spacestr      db      ` `, 0
    lbstr         db      `\n`, 0

```

Программа C:

```

#include <stdlib.h>
#include <stdio.h>

#include <stdint.h>
#include <inttypes.h>
#include <time.h>
#include <unistd.h>

void print_bin(uint16_t number);
uint16_t circular_shift(uint16_t number, int N);

// Вывод двоичного числа с пробелами
void print_bin(uint16_t number) {
    for (int i = sizeof(number) * 8 - 1; i >= 0; i--) {
        int bit = (number >> i) & 1;
        printf("%d", bit);
        if (i % 8 == 0 && i != 0) {
            printf(" ");
        }
    }
    printf("\n");
}

```



```

}

// Циклический сдвиг со свёрткой байтов
uint16_t circular_shift(uint16_t number, int N) {
    uint16_t result = 0;
    for (int i = 0; i < (int) sizeof(number); i++) {
        uint8_t byte = (number >> (i * 8)) & 0xFF;
        // Извлечение старшей и младшей тетрады
        uint8_t high_tetr = byte >> 4;
        uint8_t low_tetr = byte & 0xF;
        // Циклический сдвиг тетрад
        high_tetr = ((high_tetr >> N) | (high_tetr << (4 - N))) & 0xF;
        low_tetr = ((low_tetr << N) | (low_tetr >> (4 - N))) & 0xF;
        uint8_t new_byte = (high_tetr << 4) | low_tetr;
        result |= (new_byte << (i * 8));
    }
    return result;
}

int main(int argc, char *argv[]) {
    // Проверка запуска с переменной среды, включающей отладочный вывод.
    char *DEBUG = getenv("LAB1DEBUG");
    if (DEBUG) {
        fprintf(stderr, "Включен вывод отладочных сообщений\n");
    }

    srand(time(NULL) + getpid());
    uint16_t number;
    int N = rand() % 4;
    char *endptr;
    if (argc == 1) { // Если число не передано, то генерируем случайное
        number = rand() & UINT16_MAX;
    } else if (argc == 2) { // Если число передано, то
        number = strtoul(argv[1], &endptr, 10); // проверяем на валидность
        if (endptr == argv[1] || *endptr != '\0') {
            fprintf(stderr, "Ошибка: '%s' не является целым числом.\n", argv[1]);
            return EXIT_FAILURE;
        }
    } else {
        fprintf(stderr, "Usage: %s [число]\n", argv[0]);
        return EXIT_FAILURE;
    }

    printf("N: %d\n", N);
    print_bin(number);
    print_bin(circular_shift(number, N));

    return EXIT_SUCCESS;
}

```